



Message Passing Programming with MPI

Salah El Falou

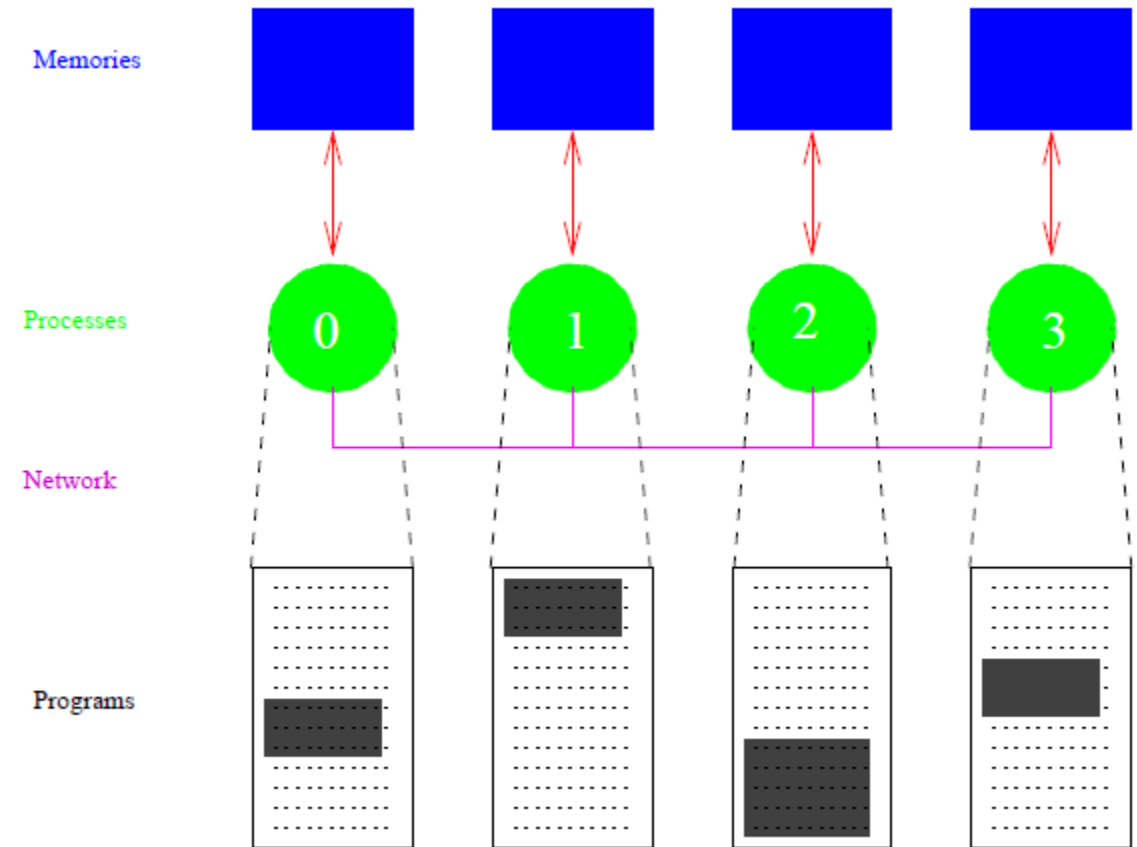
Salah.elfalou@gmail.com

What is MPI?

- First message-passing interface standard.
- Sixty people from forty different organizations.
- Users and vendors represented, from the US and Europe.
- Two-year process of proposals, meetings and review.
- *Message Passing Interface* document produced.

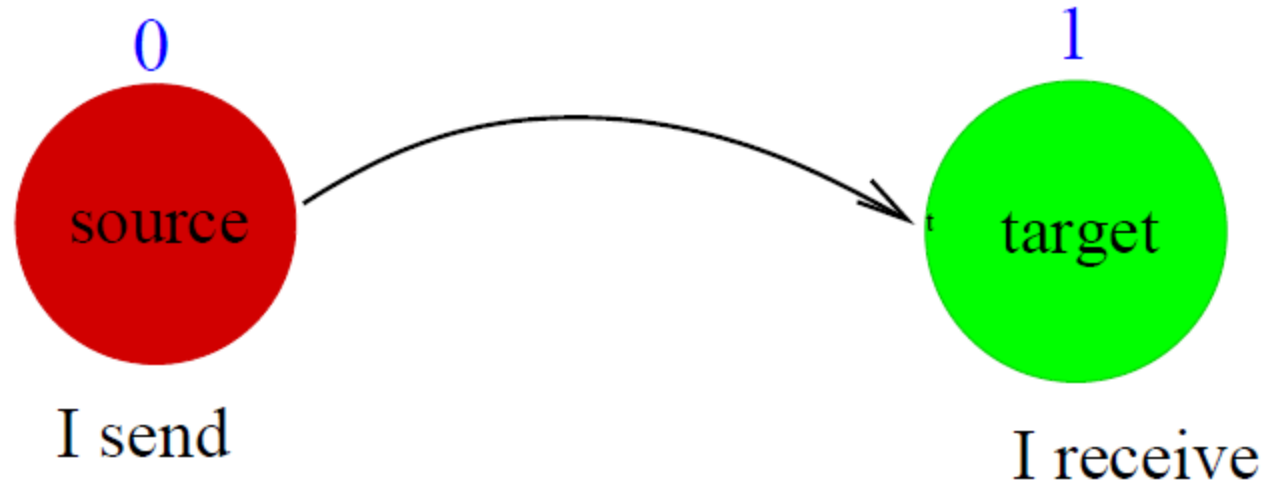
Message passing programming model

- The program is written in a classic language (Fortran, C, C++, etc.).
- All the program variables are private and reside in the local memory of each process.
- Each process has the possibility of executing different parts of a program.
- A variable is exchanged between two or several processes via a programmed call to specific subroutines.



Concept of message passing

- If a message is sent to a process, the process must receive it.



Environment

- The exchanged messages are interpreted and managed by an environment comparable to telephony, e-mail, postal mail, etc.
- The message is sent to a specified address.
- The receiving process must be able to classify and interpret the messages which are sent to it.
- The environment in question is MPI (Message Passing Interface). An MPI application is a group of autonomous processes, each executing its own code and communicating via calls to MPI library subroutines.

MPI program

➤ Header file

- `#include <mpi.h>`

➤ Initializing MPI

- `int MPI_Init(int *argc, char ***argv)`
- Must be the first MPI procedure called.

➤ Exiting MPI

- `int MPI_Finalize()`
- Must be the last MPI procedure called.

➤ MPI Function Format

- `error = MPI_Xxxxx(parameter, ...);`
- `MPI_Xxxxx(parameter, ...);`

➤ Current process id

- `MPI_Comm_rank (MPI_COMM_WORLD, &rank);`

➤ Number of processes

- `MPI_Comm_size (MPI_COMM_WORLD, &size);`

```
#include "stdio.h"
#include "mpi.h"
int main (int argc, char ** argv)
{
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf("Hello world\n");
    MPI_Finalize();
    return 0;
}
```

[[ex1 hello](#)]

Compilation and execution of an MPI code

- To compile an MPI code, we use a compiler, which makes the link with the chosen MPI library.

```
>mpicc -o ex1 ex1_hello.c
```

- To execute an MPI code, we use an MPI launcher, which runs the execution on a given number of processes.
- The `mpiexec` launcher is defined by the MPI standard. There are also non-standard launchers, such as `mpirun`.

```
>mpiexec -n 2 ex1
```

Exercise

- Write a minimal MPI program which prints ``hello world''.
- Compile it.
- Run it on a single processor.
- Run it on several processors in parallel.
- Modify your program so that only the process ranked 0 in **MPI_COMM_WORLD** prints out.
- Modify your program so that the number of processes is printed out.

Termination of a program

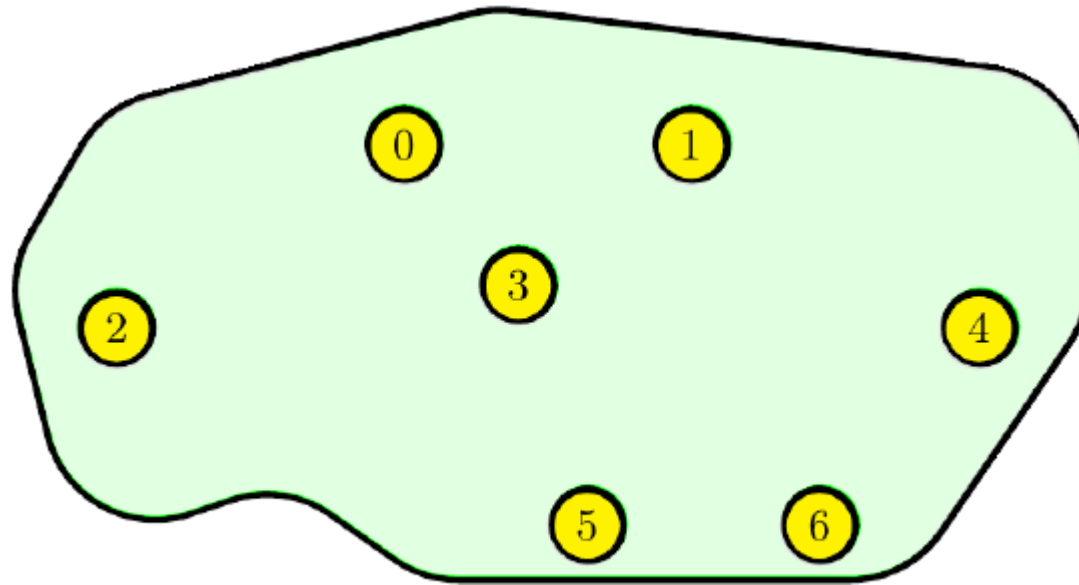
[\[ex2-abort\]](#)

- Sometimes, a program encounters some issue during its execution and has to stop prematurely.
- For example, we want the execution to stop if one of the processes cannot allocate the memory needed for its calculation.
- In this case, we call the MPI_Abort () subroutine.
- Syntax:
`MPI_Abort(MPI_COMM_WORLD,-1);`

Messages

Communicators

- All the MPI operations occur in a defined set of processes, called communicator.
- The default communicator is MPI_COMM_WORLD, which includes all the active processes.

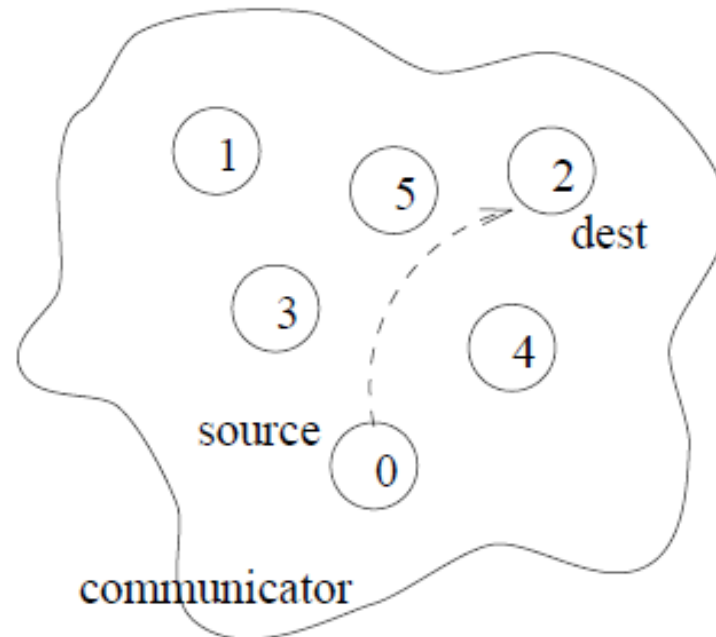


MPI_COMM_WORLD Communicator

Point-to-point Communications

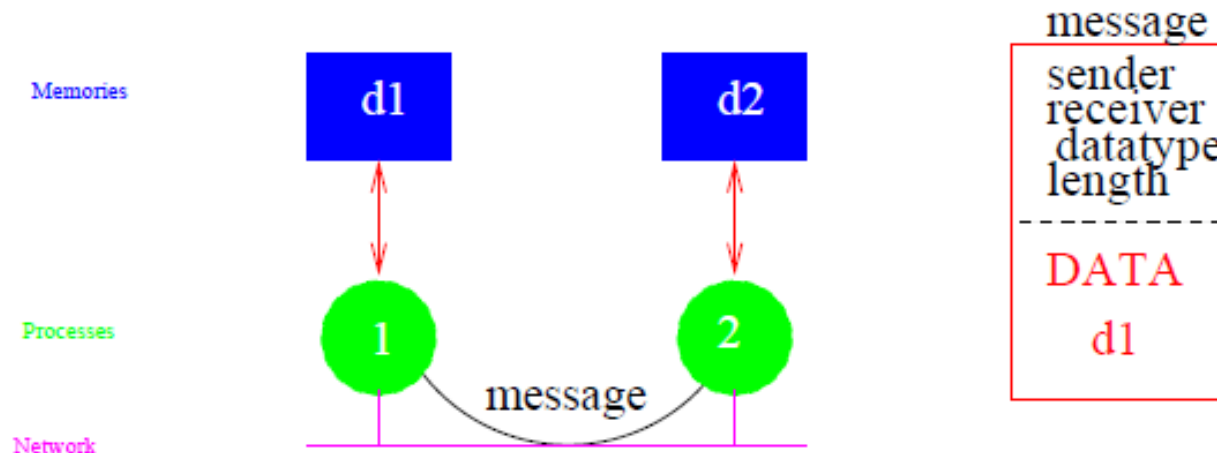
Point-to-point Communications

- Communication between two processes.
- Source process sends message to destination process.
- Communication takes place within a communicator.
- Destination process is identified by its rank in the communicator.



Message content

- A message consists of data chunks passing from the sending process to the receiving process.
- In addition to the data (scalar variables, arrays, etc.) to be sent, a message must contain the following information:
 - The identifier of the sending process
 - The datatype
 - The length
 - The identifier of the receiving process



Sending a message

- `int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`
- Sending, from the address `buf`, a message of `count` elements of type `datatype`, tagged `tag`, to the process of rank `dest` in the communicator `comm`.
- **Remark :**

This call is blocking : the execution remains blocked until the message can be re-written without risk of overwriting the value to be sent. In other words, the execution is blocked as long as the message has not been received.

Receiving a message

- `int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)`
- Receiving, at the address `buf`, a message of `count` elements of type `datatype`, tagged `tag`, from the process of rank `source` in the communicator `comm`.
- **Remarks :**
 - `status_msg` stores the state of a receive operation : source, tag, code,
 - An `MPI_Recv` can only be associated to an `MPI_Send` if these two calls have the same envelope (`source`, `dest`, `tag`, `comm`).
 - This call is blocking : the execution remains blocked until the message content corresponds to the received message.

Exercise

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <mpi.h>

int main (int argc, char ** argv)
{
    int rank, size,a,b,code,tag=100;
    MPI_Init (&argc, &argv); /* starts MPI */
    MPI_Comm_rank (MPI_COMM_WORLD, &rank); /* get current process id */

    if (rank == 0) {
        a=1000;
        code=MPI_Send(&a,1, MPI_INTEGER,1,tag, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        code=MPI_Recv(&b,1, MPI_INTEGER,0,tag, MPI_COMM_WORLD,MPI_STATUSES_IGNORE);
        printf("Process 1, has received %d from process 0.\n",b);
    }

    MPI_Comm_size (MPI_COMM_WORLD, &size); /* get number of processes */
    MPI_Finalize();
    return 0;
}
```

[[ex3SendReceive](#)]

```
>mpiexec -n 2 ex3
```

```
Process 1, has received 1000 from process 0.
```

Message Datatypes

MPI Datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Point-to-point Communications

➤ **For a communication to succeed:**

- Sender must specify a valid destination rank.
- Receiver must specify a valid source rank.
- The communicator must be the same.
- Tags must match.
- Message types must match.
- Receiver's buffer must be large enough.

Exercise

[\[ex4SendReceiveSum\]](#)

- Write a program with two processes the first one read 2 values from the keyboard and send the two values to process 2.
- Process 2 should wait until he receive the 2 values.
- Process 2 compute the sum and send the results back to process 1.
- Process 1 display the results.

Other possibilities

- When receiving a message, the rank of the sender process and the tag can be replaced by « jokers » : `MPI_ANY_SOURCE` and `MPI_ANY_TAG`, respectively.
- A communication involving the dummy process of rank `MPI_PROC_NULL` has no effect.
- `MPI_STATUS_IGNORE` is a predefined constant, which can be used instead of the status variable.
- It is possible to send more complex data structures by creating derived datatypes.
- There are other operations, which carry out both send and receive operations simultaneously : `MPI_Sendrecv()` and `MPI_Sendrecv_replace()`.

Communication Envelope Information

- Envelope information is returned from **MPI_Recv** as **status**
- Information includes:
 - Source: **status.MPI_SOURCE** or **status(MPI_SOURCE)**
 - Tag: **status.MPI_TAG** or **status(MPI_TAG)**
 - Count: **MPI_Get_count**

```
int MPI_Get_count(MPI_Status *status, MPI_Datatype  
datatype, int *count)
```

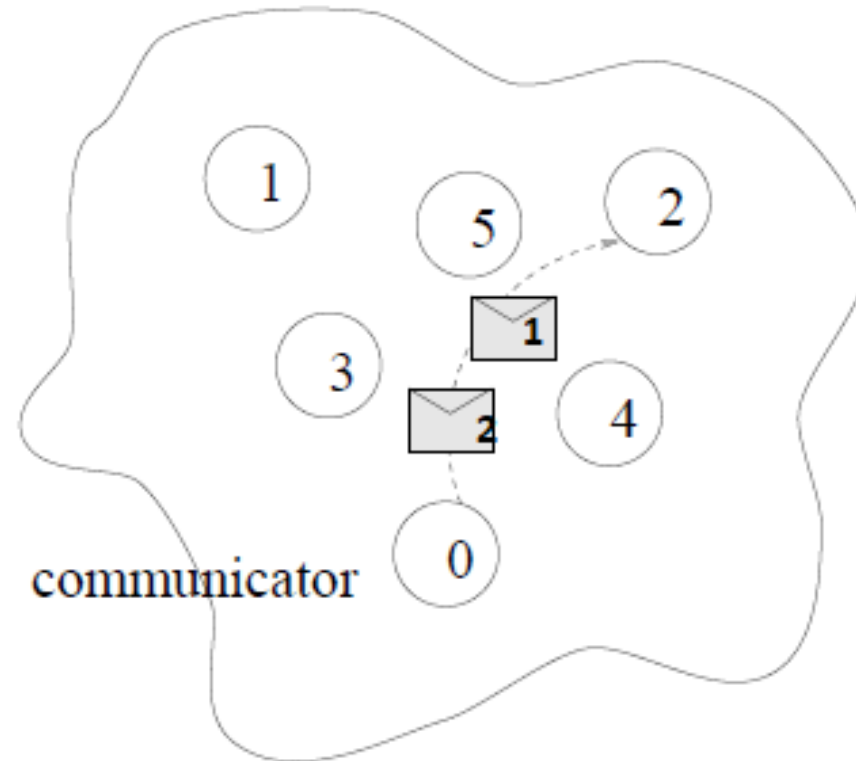
Exercise – String message

```
#include ...
#define max 100
int main (int argc, char ** argv)
{
    int rank, size, code, tag=100, count;
    MPI_Status status;
    char data [100];
    MPI_Init (&argc, &argv);          /* starts MPI */
    MPI_Comm_rank (MPI_COMM_WORLD, &rank); /* get current process id */
    if (rank == 0) {
        printf("Message=?\n");
        scanf("%s", data);
        code=MPI_Send(data, strlen(data), MPI_CHAR, 1, tag, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        code=MPI_Recv(data, 100, MPI_CHAR, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        MPI_Get_count(&status, MPI_CHAR, &count);
        printf("Source=%d tag=%d count=%d\n", status.MPI_SOURCE, status.MPI_TAG, count);
        printf("Me, process 1, i have received %s from process %d.\n", data, status.MPI_SOURCE);
    }
    MPI_Finalize();
    return 0;
}
```

[[Ex5SendReceiveString](#)]

Message Order Preservation

- Messages do not overtake each other.
- This is true even for non-synchronous sends.



Timers

```
double MPI_Wtime(void) ;
```

- Time is measured in seconds.
- Time to perform a task is measured by consulting the timer before and after.
- Modify your program to measure its execution time and print it out.

Exercise - Ping pong

- Write a program in which the process 0 sends a message containing 1000 random reals to process 1.
- Insert timing calls to measure the time taken for 1000 messages.
- Write another program in such a way that the process 1 sends back the message to the process 0, and measure the communication duration with the `MPI_Wtime()` function.
- Investigate how the time taken varies with the size of the message.

Non-Blocking Communications

Deadlock (1/2)

- When two processes exchange messages of large sizes, the execution of a program calling the MPI_Send function followed by MPI_Recv will remain blocked on MPI_Send.
- Large messages are sent in synchronous mode. This synchronous mode implies that in order for the message to leave (the MPI_Send call can be terminated), the reception of this message must have been posted (the call to MPI_Recv has been made).

Process 0	Process 1
Send(1)	Send(0)
Recv(1)	Recv(0)

- This communication method is not recommended at all and is also considered erroneous by the MPI standard.

Deadlock (2/2)

- The scenario that avoids the deadlock phenomenon:

Process 0	Process 1
Send(1)	Recv(0)
Recv(1)	Send(0)

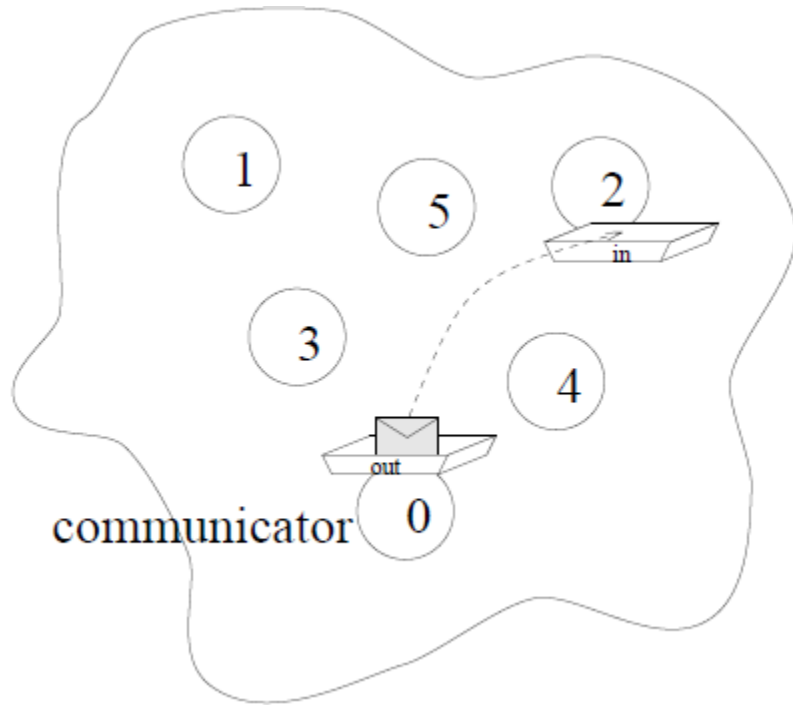
- Beware of tags: if a process of rank R1 sends a message M to the process R2 with the tag E1, the process R2 receives the message M with the same tag E1. Even with this scenario, if the tags are not well put, it can still block!

Non-Blocking Communications

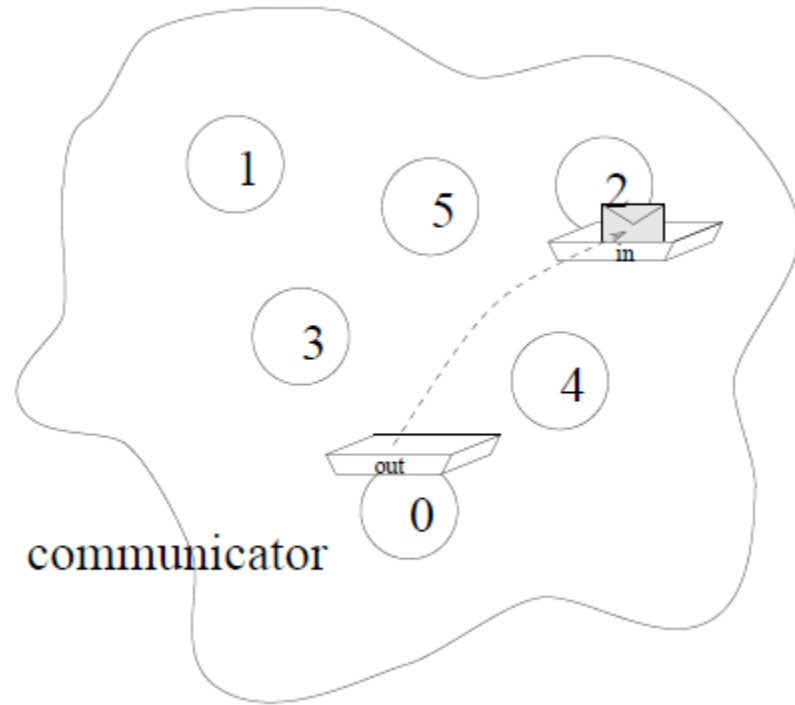
- Separate communication into three phases:
- Initiate non-blocking communication.
- Do some work (perhaps involving other communications?)
- Wait for non-blocking communication to complete.

Non-Blocking Send/Receive (1/2)

Non-Blocking Send



Non-Blocking Receive



Non-Blocking Send/Receive (2/2)

[\[ex3SendReceiveNb\]](#)

➤ Non-blocking Synchronous Send

```
int MPI_Issend(void* buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm,  
MPI_Request *request)
```

```
int MPI_Wait(MPI_Request *request, MPI_Status *status)
```

➤ Non-Blocking Receive

```
int MPI_Irecv(void* buf, int count, MPI_Datatype datatype, int src, int tag,  
MPI_Comm comm, MPI_Request *request)
```

```
int MPI_Wait(MPI_Request *request, MPI_Status *status)
```


Blocking and Non-Blocking

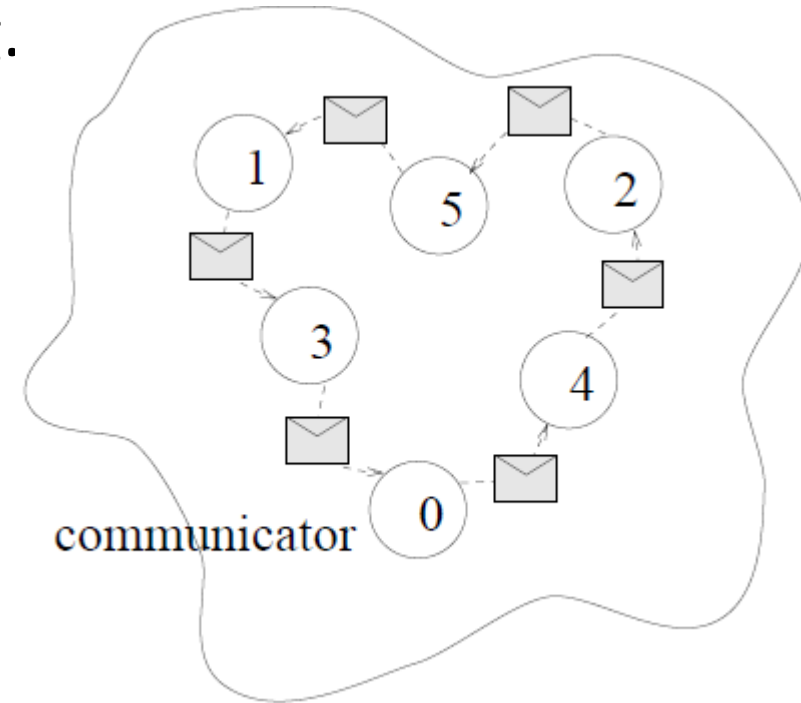
- Send and receive can be blocking or non-blocking.
- A blocking send can be used with a non-blocking receive, and vice-versa.
- Non-blocking sends can use any mode - synchronous, buffered, standard, or ready.
- Synchronous mode affects completion, not initiation.
- A synchronous send involves a synchronization between the involved processes. A send cannot start until its receive is posted.

Point-to-Point Send Modes

<i>Mode</i>	Blocking	Non-blocking
Standard send	<code>MPI_SEND ()</code>	<code>MPI_ISEND ()</code>
Synchronous send	<code>MPI_SSEND ()</code>	<code>MPI_ISSEND ()</code>
Buffered send	<code>MPI_BSEND ()</code>	<code>MPI_IBSEND ()</code>
Receive	<code>MPI_RECV ()</code>	<code>MPI_Irecv ()</code>

Exercise

- **Rotating information around a ring**
- Arrange processes to communicate around a ring.
- Each process stores a copy of its rank in an integer variable.
- Each process communicates this value to its right neighbor, and receives a value from its left neighbor.
- Each process computes the sum of all the values received.
- Repeat for the number of processes involved and print out the sum stored at each process.



Collective Communications

General concepts

- **Collective** communications allow making a series of point-to-point communications in one single call.
- A collective communication always concerns all the processes of the indicated **communicator**.
- For each process, the call ends when its participation in the collective call is completed, in the sense of point-to-point communications (therefore, when the concerned memory area can be changed).
- The management of **tags** in these communications is transparent and system-dependent. Therefore, they are never explicitly defined during calls to subroutines. An advantage of this is that collective communications never interfere with point-to-point communications.

Types of collective communications

There are three types of subroutines :

1. One which ensures global synchronizations : `MPI_Barrier()`
2. Ones which only transfer data :
 - Global distribution of data : `MPI_Bcast()`
 - Selective distribution of data : `MPI_Scatter()`
 - Collection of distributed data : `MPI_Gather()`
 - Collection of distributed data by all the processes : `MPI_Allgather()`
 - Collection and selective distribution by all the processes of distributed data : `MPI_Alltoall()`
3. Ones which, in addition to the communications management, carry out operations on the transferred data :
 - Reduction operations (sum, product, maximum, minimum, etc.), whether of a predefined or personal type : `MPI_Reduce()`
 - Reduction operations with distributing of the result (this is in fact equivalent to an `MPI_Reduce()` followed by an `MPI_Bcast()`) : `MPI_Allreduce()`

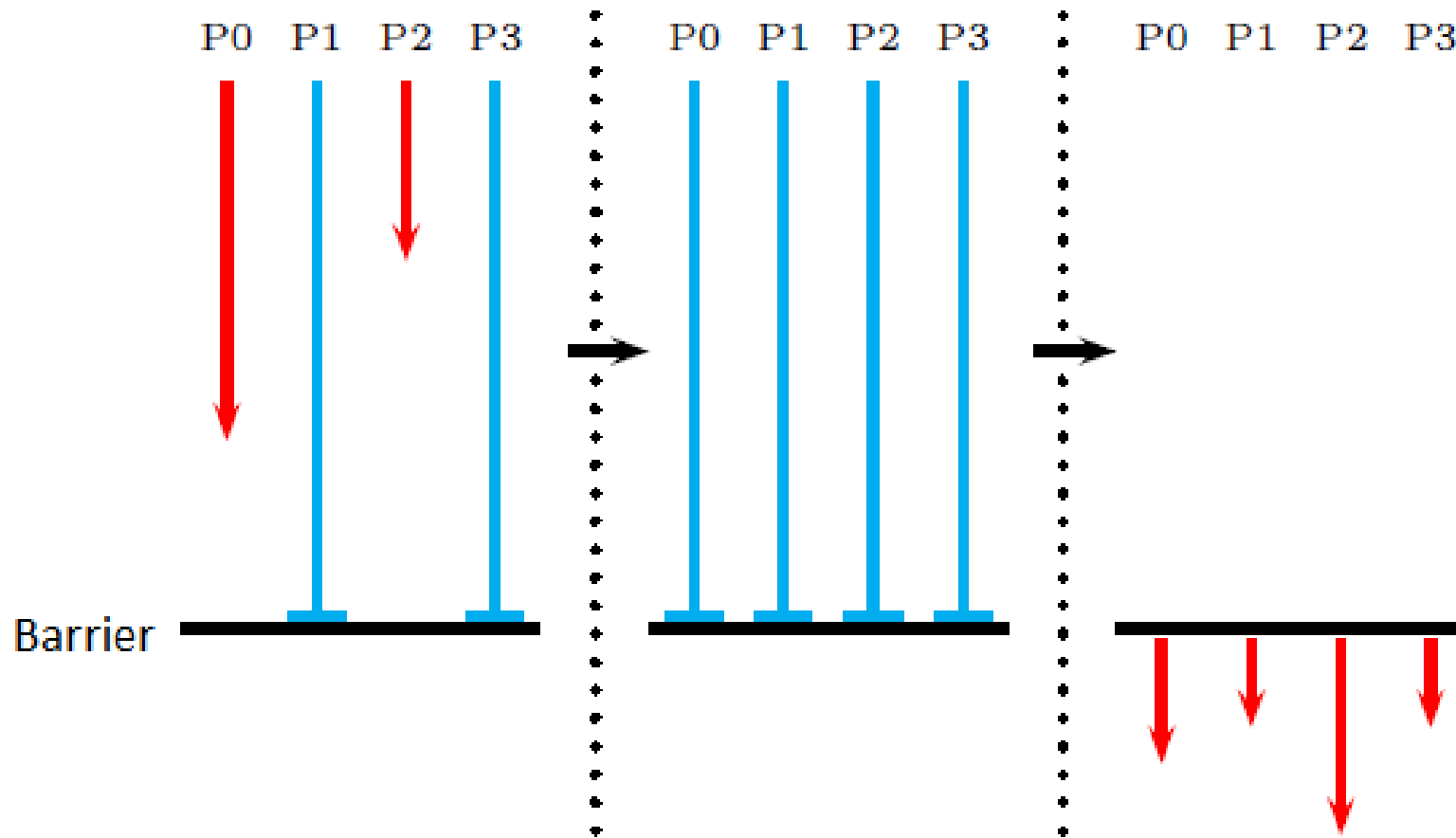
Characteristics

- Collective action over a communicator.
- All processes must communicate.
- Synchronization may or may not occur.
- All collective operations are blocking.
- No tags.
- Receive buffers must be exactly the right size.

Global synchronization : **MPI_Barrier**

Syntax:

```
int MPI_Barrier (MPI_Comm comm)
```



Exercise : MPI_Barrier

[\[Ex6Barrier\]](#)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <mpi.h>
int main(int argc, char *argv[])
{
    int rank, nprocs;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if(rank==0 )
    {
        printf("Start process %d\n", rank); fflush(stdout);
        sleep(1);
        printf("End process %d\n", rank); fflush(stdout);
    }
    else if(rank==1)
    {
        printf("Start process %d\n", rank); fflush(stdout);
        sleep(5);
        printf("End process %d\n", rank); fflush(stdout);
    }
    MPI_Barrier(MPI_COMM_WORLD);
    printf("After Barrier I am %d of %d\n", rank, nprocs); fflush(stdout);
    MPI_Finalize();
    return 0;
}
```


Exercise

- Write a MPI program that compute the sum:

[\[ex7BarrierSum\]](#)

$$S=1+2+3 \dots +n$$

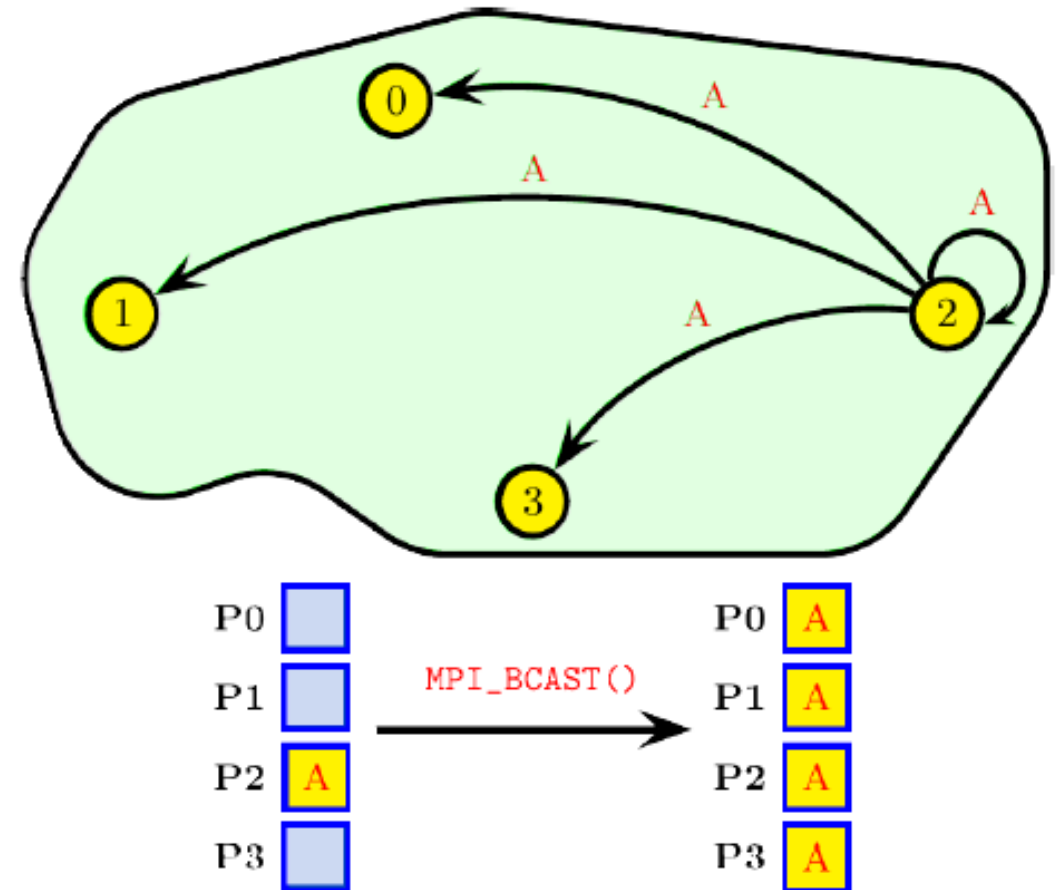
- Consider 2 processes where the sum is calculated in parallel, each process compute a local sum.
- Add Barrier to synchronize before computing the correct sum.
- To have a correct answer we should communicate in order to exchange local sum.
- Using blocking send/receive we don't need the barrier.

Broadcast

```
int MPI_Bcast (void *buffer, int count, MPI_Datatype  
datatype, int root, MPI_Comm comm)
```

[\[Ex8Broadcast\]](#)

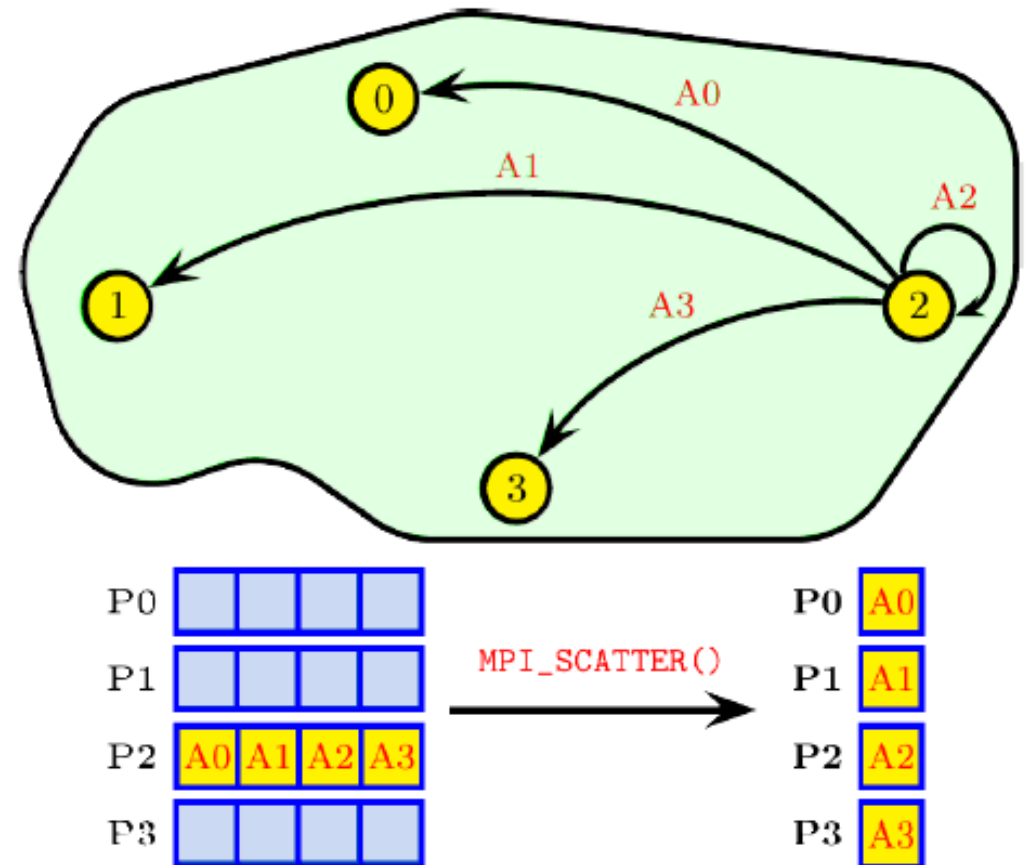
- Send, starting at position **buffer**, a message of **count** element of type **datatype**, by the **root** process, to all the members of communicator **comm**.
- Receive this message at position **buffer** for all the processes other than the root.



Selective distribution : MPI_Scatter (1/2)

```
int MPI_Scatter(void *sendbuf,int sendcount, MPI_Datatype  
sendtype, void *recvbuf, int recvcount, MPI_Datatype  
recvtype, int root,MPI_Comm comm)
```

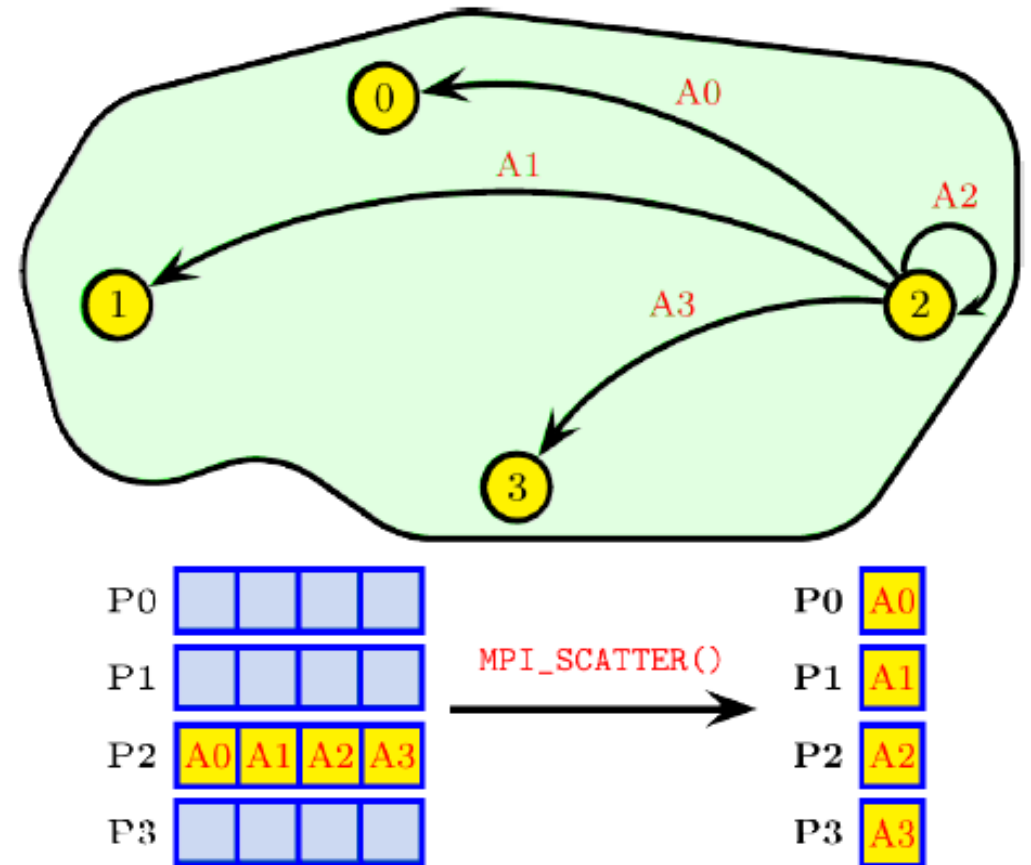
- Scatter by process **root**, starting at position **sendbuf**, message **sendcount** element of type **sendtype**, to all the processes of communicator **comm**.
- Receive this message at position **recvbuf**, of **recvcount** element of type **recvtype** for all processes of communicator **comm**.



Selective distribution : MPI_Scatter (2/2)

Remarks :

- The couples (sendcount, sendtype) and (recvcount, recvtype) must represent the same quantity of data.
- Data are scattered in chunks of same size ; a chunk consists of sendcount elements of type sendtype.
- The i-th chunk is sent to the i-th process.



Exercise

```
int main(int argc, char *argv){
    int rank, nprocs,block_length,nb_values=8;
    int * data,* values;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    block_length=nb_values/nprocs;
    data=(int*)malloc(block_length*sizeof(int));
    values=(int*)malloc(nb_values*sizeof(int));
    if (rank == 2){
        printf("Process %d, send the array values:",rank);fflush(stdout);
        for(int i=0;i<nb_values;i++){
            values[i]=1000+i;
            printf("%d,",values[i]);fflush(stdout);
        }
        printf("\n");fflush(stdout);
    }
    MPI_Scatter(values,block_length, MPI_REAL,data,block_length, MPI_REAL,2, MPI_COMM_WORLD);
    printf("Process %d received data ",rank);
    for(int i=0;i<block_length;i++)
        printf("%d,",data[i]);fflush(stdout);
    printf("from process 2\n");fflush(stdout);
    MPI_Finalize();
    return 0;
}
```

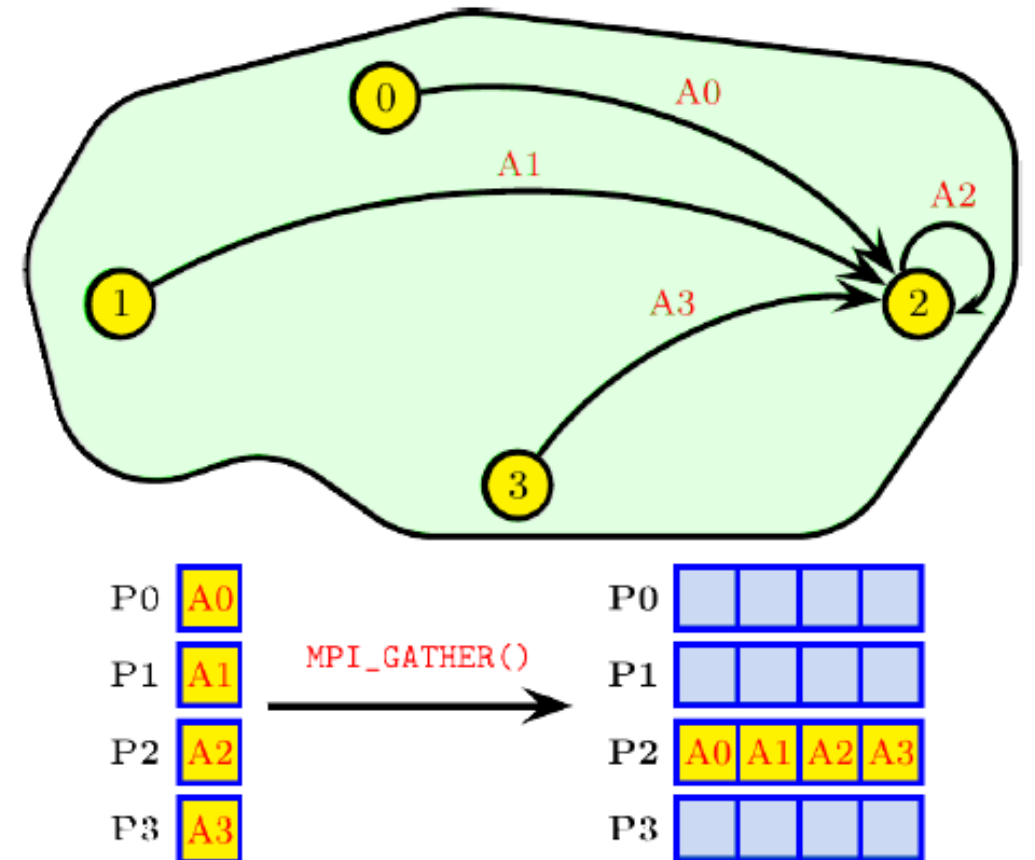
[Ex9Scatter]

```
salah@ubuntu:~/ufaz/mpi$ mpiexec -n 4 ex9
Process 2, send the array values:1000,1001,1002,1003,1004,1005,1006,1007,
Process 0 received data 1000,1001,from process 2
Process 2 received data 1004,1005,from process 2
Process 1 received data 1002,1003,from process 2
Process 3 received data 1006,1007,from process 2
```

MPI_Gather (1/2)

```
int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype  
sendtype, void *recvbuf, int recvcount, MPI_Datatype  
recvtype, int root, MPI_Comm comm)
```

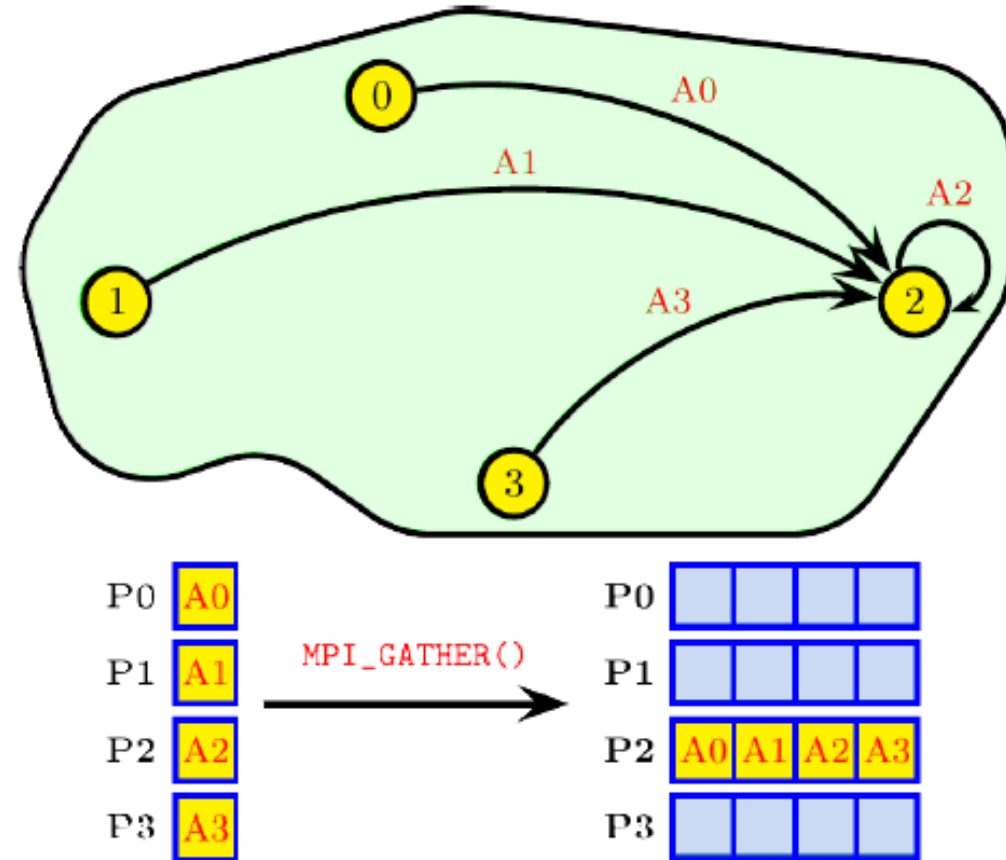
- Send for each process of communicator **comm**, a message starting at position **sendbuf**, of **sendcount** element type **sendtype**.
- Collect all these messages by the **root** process at position **recvbuf**, **recvcount** element of type **recvtype**.



MPI_Gather (2/2)

Remarks :

- The couples (sendcount, sendtype) and (recvcount, recvtype) must represent the same size of data.
- The data are collected in the order of the process ranks.
- The i-th process send the i-th chunk.



Exercise

```
int main(int argc, char *argv[]) {
    int rank, nprocs, block_length, nb_values=8;
    int * data, * values;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    block_length = nb_values / nprocs;
    data = (int*) malloc(nb_values * sizeof(int));
    values = (int*) malloc(block_length * sizeof(int));
    for(int i=0; i<block_length; i++) {
        values[i] = 1000 + i + (rank * block_length);
        printf("Process %d send %d\n", rank, values[i]); fflush(stdout);
    }
    MPI_Gather(values, block_length, MPI_REAL, data, block_length, MPI_REAL, nprocs-1, MPI_COMM_WORLD);
    if (rank == nprocs-1) {
        printf("Me, Process %d i have received data: ", rank);
        for(int i=0; i<nb_values; i++)
            printf("%d, ", data[i]); fflush(stdout);
        printf("\n");
    }
    MPI_Finalize();
    return 0;
}
```

[Ex10Gather]

```
Process 0 send 1000
Process 0 send 1001
Process 2 send 1004
Process 2 send 1005
Process 1 send 1002
Process 1 send 1003
Process 3 send 1006
Process 3 send 1007
Me, Process 3 i have received data: 1000,1001,1002,1003,1004,1005,1006,1007,
```


Exercise

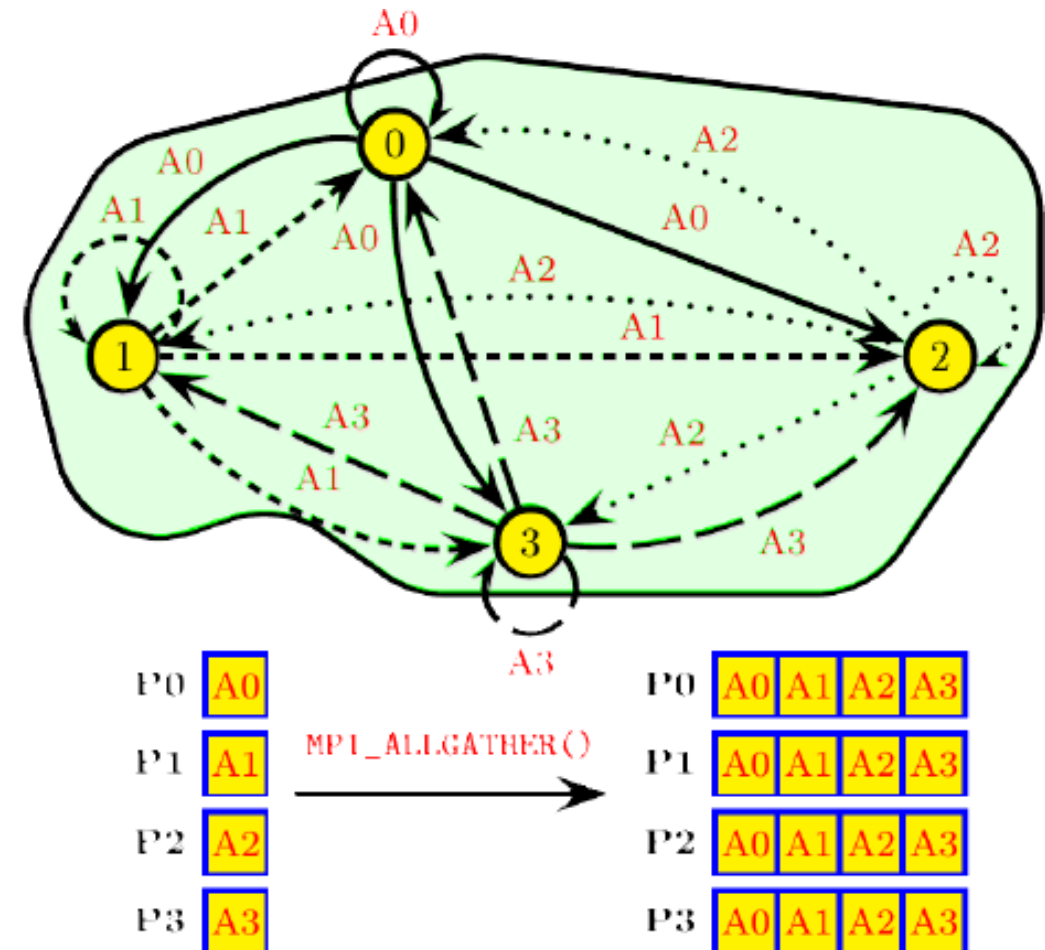
[\[Ex11ScatGathSum\]](#)

- Write a parallel MPI program that compute the sum of elements in a table.
- The table is distributed using MPI_Scatter.
- Each process compute a local sum.
- Results are collected by process 0 using MPI_Gather.
- Process 0 compute the global sum.

MPI_Allgather (1/2)

```
int MPI_Allgather(const void * sendbuf, int sendcount,  
MPI_Datatype sendtype, void * recvbuf, int recvcount, MPI_Datatype  
recvtype, MPI_Comm comm)
```

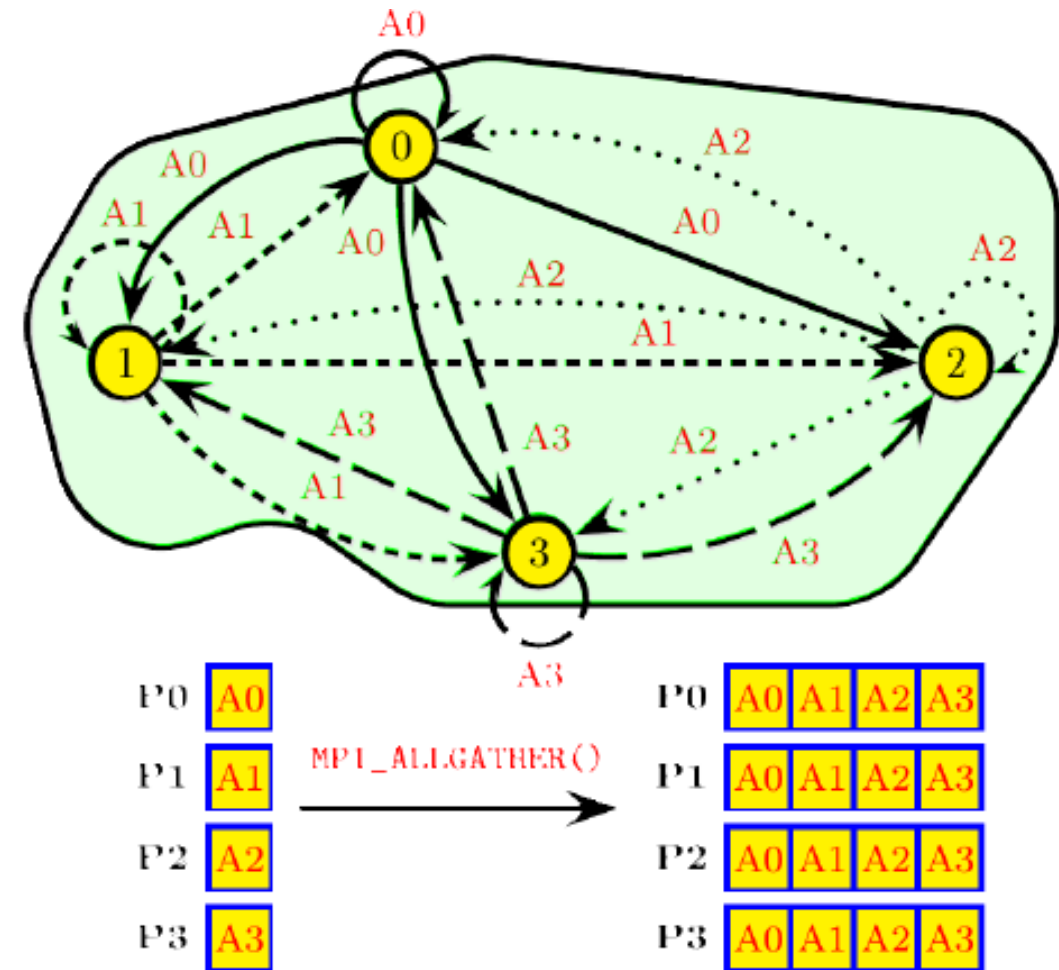
- Send by each process of communicator **comm**, a message starting at position **sendbuf**, of **sendcount** element, type **sendtype**.
- Collect all these messages, by all the processes, at position **recvbuf** of **recvcount** element type **recvtype**.



MPI_Allgather (2/2)

Remarks :

- The couples (sendcount, sendtype) and (recvcount, recvtype) must represent the same data size.
- The data are gathered in the order of the process ranks.
- The i-th process send the i-th chunk.

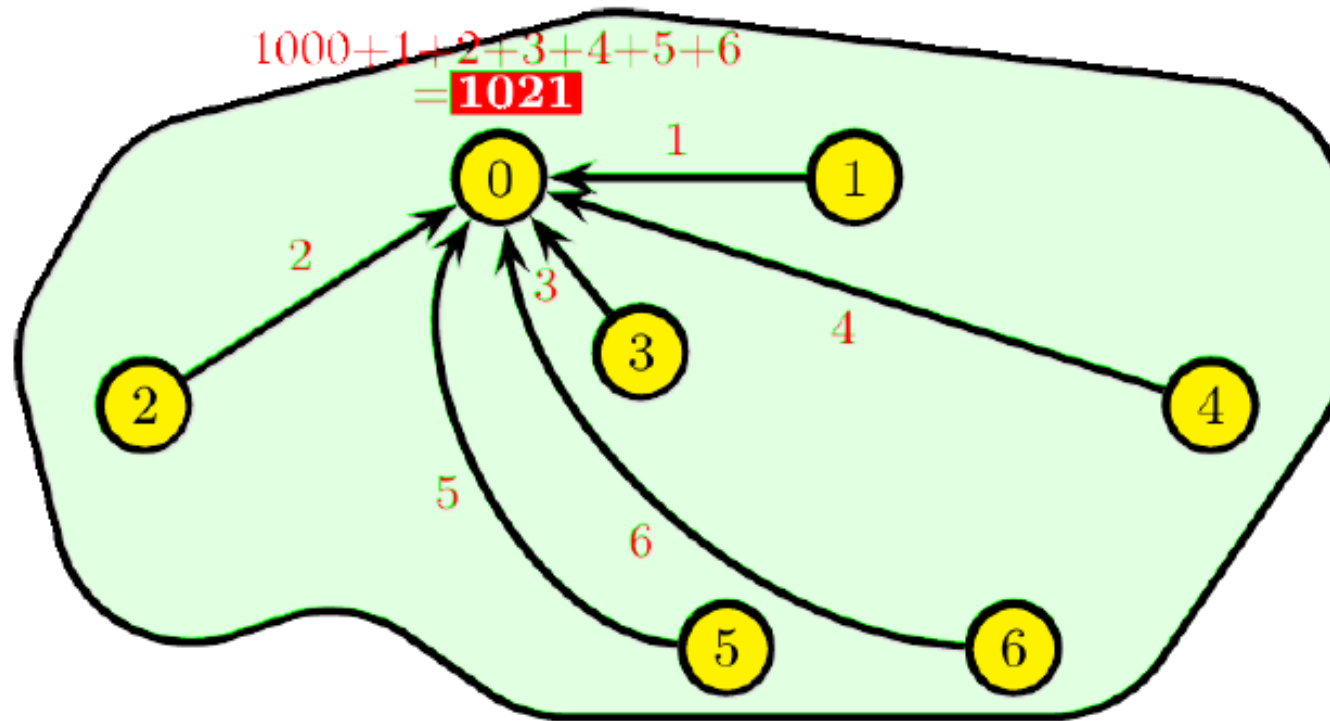


Global Reduction Operations

- A reduction is an operation applied to a set of elements in order to obtain one single value. Typical examples are the sum of the elements of a vector (`SUM(A(:))`) or the search for the maximum value element in a vector (`MAX(V(:))`).
- MPI proposes high-level subroutines in order to operate reductions on data distributed on a group of processes. The result is obtained on only one process (`MPI_Reduce()`) or on all the processes (`MPI_Allreduce()`), which is in fact equivalent to an `MPI_Reduce()` followed by an `MPI_Bcast()`.
- If several elements are implied by process, the reduction function is applied to each one of them (for instance to each element of a vector).

Reduction

- Used to compute a result involving data distributed over a group of processes.



Distributed reduction (sum)

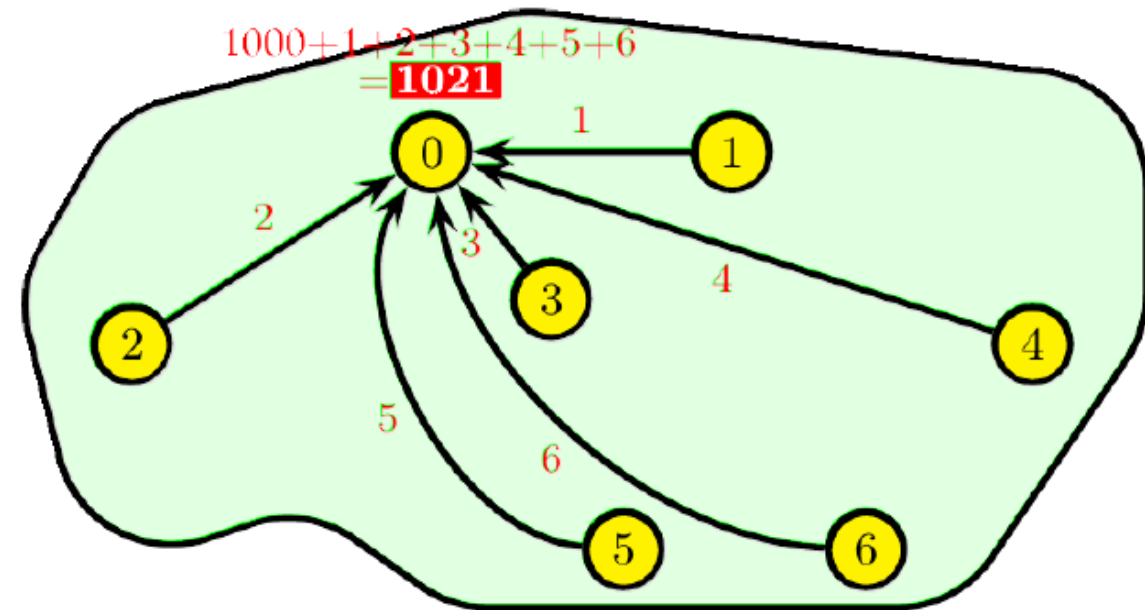
Predefined Reduction Operations

Name	Operation
<code>MPI_SUM</code>	Sum of elements
<code>MPI_PROD</code>	Product of elements
<code>MPI_MAX</code>	Maximum of elements
<code>MPI_MIN</code>	Minimum of elements
<code>MPI_MAXLOC</code>	Maximum of elements and location
<code>MPI_MINLOC</code>	Minimum of elements and location
<code>MPI LAND</code>	Logical AND
<code>MPI_LOR</code>	Logical OR
<code>MPI_LXOR</code>	Logical exclusive OR

MPI_Reduce

```
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,  
MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

- Distributed reduction of **count** elements of type **datatype**, starting at position **sendbuf**, with the operation **op** from each process of the communicator **comm**



Distributed reduction (sum)

Exercise

```
int main (int argc, char ** argv)
{
    int rank, size, sum, code;
    int * data;
    MPI_Init (&argc, &argv);          /* starts MPI */
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);          /* get current process id */
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    data=(int *)malloc(size*sizeof(int));
    data[rank]=rank;
    MPI_Reduce (&data[rank], &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0)
    {
        printf("Me, procces 0, Sum = %d \n", sum);
    }
    MPI_Finalize();
    return 0;
}
```

[[Ex12ReductionSum](#)]

```
salah@ubuntu:~/ufaz/mpi$ mpirun -n 4 ex12
Me, procces 0, Sum = 6
```


Exercise

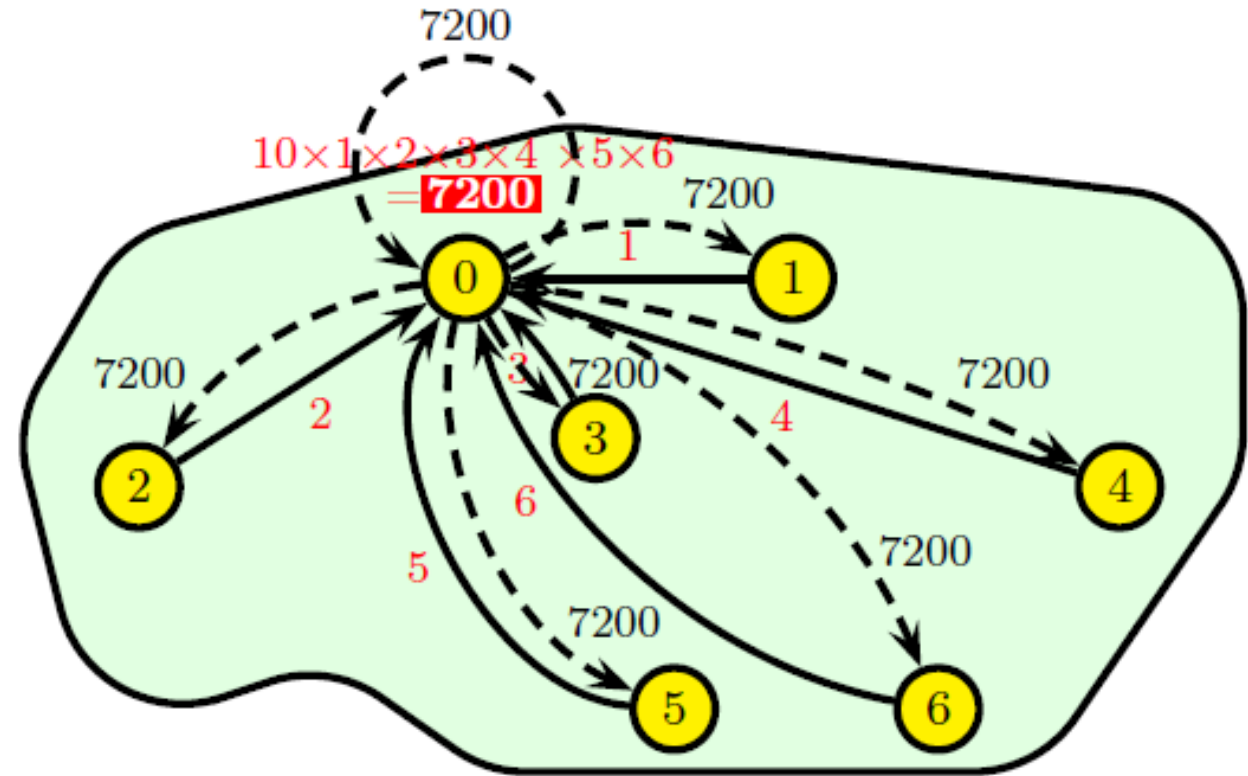
[\[Ex13ScatRedSum\]](#)

- Re-Write the parallel program that compute the sum of elements in a table.
- The table is distributed using MPI_Scatter.
- Each process compute a local sum.
- Results are collected by process 0 using MPI_Reduce.
- Process 0 display the sum.

MPI_Allreduce

```
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,  
MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

- Distributed reduction of **count** elements of type **datatype**, starting at position **sendbuf**, with the operation **op** from each process of the communicator **comm**.
- Write the result at position **recvbuf** for all the processes of the communicator **comm**.



Exercise

```
int main (int argc, char ** argv)
{
    int rank, size, sum, code;
    int * data;
    MPI_Init (&argc, &argv); /* starts MPI */
    MPI_Comm_rank (MPI_COMM_WORLD, &rank); /* get current process id */
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    data=(int *)malloc(size*sizeof(int));
    data[rank]=rank;
    MPI_Allreduce (&data[rank], &sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    printf("Me, procces %d, Sum = %d \n", rank, sum);
    MPI_Finalize();
    return 0;
}
```

[[Ex14AllReduce](#)]

```
salah@ubuntu:~/ufaz/mpi$ mpirun -n 4 ex14
```

```
Me, procces 0, Sum = 6
```

```
Me, procces 0, Sum = 6
```

```
Me, procces 0, Sum = 6
```

```
Me, procces 0, Sum = 6
```